

Supporting Data and Code: Reproducibility Documentation

Investment concentration and financing in the AI boom

Companion to the manuscript submission

1 Purpose and scope

This document describes the supporting data-and-code archive that accompanies the manuscript. The archive is a self-contained, **offline**, **deterministic** reproduction: from a single command it regenerates every reported number, table and figure from frozen inputs, with **no network access, no API keys and no paid calls**. The permanent citable version is the Zenodo deposit at 10.5281/zenodo.22392416 (code MIT-licensed). That is the concept DOI: it represents all versions of the archive and always resolves to the latest.

A strict wall separates two phases. An *offline analysis* reproduces every claim from the frozen inputs (documented here). A *collection* phase originally queried public SEC and BLS endpoints and is retained for transparency only (Section 9). Because the SEC's *frames* interface returns the most recently filed value for each fact, re-collecting today would pick up subsequent filings or revised presentations and would not reproduce the frozen inputs; collection is therefore deliberately not part of the reproducible core.

2 Quick start

```
pip install -r requirements.txt
./verify.sh
```

Expected result: the archive verifies against its SHA-256 manifest, the analysis re-runs from the frozen inputs, the regenerated `claims.json` and figure are byte-identical to the committed ones, every number in the manuscript reconciles to `claims.json`, the manuscript is within the journal's length limit, and the script exits 0. Total runtime is under ten seconds on a standard laptop. No network access is required at any point.

3 Archive layout

<code>verify.sh</code>	one-command offline verification (referee path; Section 8)
<code>reproduce.sh</code>	same pipeline plus rebuilding <code>manuscript.pdf</code> (author path)
<code>MANIFEST.sha256</code>	SHA-256 of every archived file, in <code>sha256sum-c</code> format
<code>requirements.txt</code>	<code>matplotlib</code> (figure only); every number needs the standard library alone
<code>environment.txt</code>	exact versions that produced the committed results
<code>manuscript.tex</code> / <code>.pdf</code>	the submission (elsarticle; reproducible source)
<code>manuscript_blinded.tex</code> / <code>.pdf</code>	blinded review copy (identifiers and metadata removed)
<code>COVER_LETTER.md</code> / <code>.tex</code> / <code>.pdf</code>	cover letter
<code>highlights.txt</code>	journal highlights (5 bullets)
<code>figures/Figure_1.pdf</code> / <code>.png</code>	the figure, vector and 200-dpi raster

<code>claims.json</code>	canonical single source of truth: every number the paper asserts
<code>code/</code>	collectors, engine, claims builder, figure builder, gates (Section 6)
<code>data/frozen/</code>	8 frozen source files, read-only (Section 4)
<code>data/SHA256SUMS.txt</code>	SHA-256 of every frozen source file
<code>data/SOURCE_URLS.txt</code>	the exact URL behind every frozen data point, one line per call
<code>README.md</code> , <code>DATA_AVAILABILITY.md</code>	orientation and the data statement
<code>CITATION.cff</code> , <code>.zenodo.json</code> , <code>LICENSE</code>	citation and deposit metadata, licences

4 Data (inputs and metadata)

4.1 Frozen inputs

Every input the pipeline reads is a frozen artifact. The analysis is a pure function of these files; it contains no random component and no seed.

File	Records	Unit of observation / contents
<code>capex_frames.json</code>	88,097	firm-year capital expenditure, 2009–2025, two tags: <code>PaymentsToAcquirePropertyPlantAndEquipment</code> , <code>PaymentsToAcquireProductiveAssets</code>
<code>composition_frames.json</code>	161,802	firm-year operating cash flow, R&D, repurchases, dividends, 2013–2025
<code>revenue_frames.json</code>	95,605	firm-year revenue (three tags) and continuing-operations cash flow, 2013–2025; revenue ranks the universe
<code>financing_frames.json</code>	63,293	firm-year long-term-debt issuance and repayment, and cash paid for acquisitions, 2013–2025
<code>alt_frames.json</code>	270,979	alternate tags for every key series (nine tags), 2013–2025; the union is what makes the measure tag-robust
<code>ai_intensity.json</code>	285	one scored 10-K per firm: dictionary occurrences, total words, AI intensity, filing period
<code>sic_codes.json</code>	500	registrant SIC code and description, as filed
<code>deflator_WPUFD41312.json</code>	13	BLS annual index values, 2013–2025

4.2 Provenance and integrity

`data/SHA256SUMS.txt` fixes every frozen source file; `MANIFEST.sha256` fixes every file in the archive, including the code and the manuscript. The collection scripts write their outputs read-only and refuse to overwrite an existing frozen file, so a silent change to an input is detectable before the analysis runs. `verify.sh` checks both manifests before it runs anything.

4.3 Data sources, with the exact URL behind every figure

All inputs are public and free. The SEC requires no API key, only a descriptive `User-Agent` header (<https://www.sec.gov/search-filings/edgar-application-programming-interfaces>). Firm-level data were retrieved recently and frozen; the archive is built from those frozen copies.

The file `data/SOURCE_URLS.txt` lists the exact URL behind every frozen data point — one line per call actually made, with the frozen file it landed in. It covers 294 XBRL *frames* calls (one per tag and calendar year), 500 submissions-API calls (one per registrant), the 285 scored 10-K filings, and the BLS series. Any line can be pasted into a browser.

Source	Endpoint, and one working example
Firm financials	<a href="https://data.sec.gov/api/xbrl/frames/us-gaap/<tag>/USD/CY<year>.json">https://data.sec.gov/api/xbrl/frames/us-gaap/<tag>/USD/CY<year>.json e.g. capital expenditure for 2025: https://data.sec.gov/api/xbrl/frames/us-gaap/PaymentsToAcquirePropertyPlantAndEquipment/USD/CY2025.json
Registrant SIC and 10-K location	<a href="https://data.sec.gov/submissions/CIK<10-digit-CIK>.json">https://data.sec.gov/submissions/CIK<10-digit-CIK>.json e.g. Alphabet (CIK 1652044): https://data.sec.gov/submissions/CIK0001652044.json
10-K documents	<a href="https://www.sec.gov/Archives/edgar/data/<CIK>/<accession>/<document>">https://www.sec.gov/Archives/edgar/data/<CIK>/<accession>/<document>
Single-firm audit trail	<a href="https://data.sec.gov/api/xbrl/companyconcept/CIK<CIK>/us-gaap/<tag>.json">https://data.sec.gov/api/xbrl/companyconcept/CIK<CIK>/us-gaap/<tag>.json used by <code>reconcile.py</code> ; shows every filing that reported a given fact
Deflator	BLS series WPUFD41312, final demand–private capital equipment, annual averages, via the BLS public timeseries API https://api.bls.gov/publicAPI/v2/timeseries/data/ (no key required at this volume); collector code/ <code>collect_deflator.py</code> , whose <code>--check</code> option compares the live series against the freeze. Series page: https://data.bls.gov/timeseries/WPUFD41312

5 Tracing a reported figure to its source

A reviewer who wants to check a number rather than re-run the pipeline can go from any figure in the paper to the SEC endpoint it came from in one command. `code/trace.py` prints every tag the series unions, the frozen value under each, the exact URL, which tag wins the union, and the deflated figure the paper uses.

```
python3 code/trace.py --firm Alphabet --series capex --year 2025
```

```
Alphabet Inc. (CIK 1652044) capital expenditure fiscal year 2025
PaymentsToAcquirePropertyPlantAndEquipment $91.447bn
  https://data.sec.gov/api/xbrl/frames/us-gaap/PaymentsToAcquirePropertyPlantAndEqui
  pment/USD/CY2025.json
PaymentsToAcquireProductiveAssets not filed
```

```
union takes the maximum across tags -> $91.447bn (nominal)
deflated to 2025 dollars (WPUFD41312, 224.732/224.732): $91.447bn
```

Adding `--live` re-fetches from the SEC and reports whether the live value still matches the frozen copy, printing the accession number of the filing that supplies it. `--list` shows the eight series available and the five focal firms. A live difference reflects a subsequent filing or revised presentation since the freeze, not an error; it is precisely why the archive is built from the frozen copies and why the paper reports them.

Worked example: the 117% at the top five (Table 1). The five largest incremental investors are Alphabet, Amazon, Oracle, Meta and Microsoft. For each, the paper cumulates the 2022–2025 increment of capital expenditure and of operating cash flow against a 2021 baseline, in 2025 dollars, and divides the second sum by the first. Every component is one `trace.py` call:

```
for f in Alphabet Amazon Oracle Meta Microsoft; do \
python3 code/trace.py --firm $f --series capex --year 2025 --live; done
```

The firm-level increments and the resulting coverage are printed in the paper’s Table A3, and the group figure is `claims.json` key `cumulative.five_A`.

6 Code (what it consumes and runs)

6.1 Analysis scripts

All code is in `code/`; each module is deterministic and reads only frozen data.

Script	Purpose	Produces
<code>engine.py</code>	measurement engine: unions alternate XBRL tags, deflates to 2025 dollars, aggregates firms; defines the tag lists for every series	imported by the others
<code>make_claims.py</code>	the whole analysis: coverage ladder, placebo windows, sample funnel, sectoral decomposition, leave-one-out, AI-intensity cuts, concentration	<code>claims.json</code>
<code>make_figure.py</code>	the three panels of Figure 1 from the same frozen inputs	<code>figures/Figure_1.pdf</code> , <code>.png</code>
<code>build_gate.py</code>	reads every number out of the manuscript and requires each to be reproducible from <code>claims.json</code> ; anchors headline figures and both table bodies to their source; cross-checks quantities reported twice	pass / fail
<code>wordcount.py</code>	journal length count (main text, table notes, figure captions, references)	pass / fail
<code>make_manifest.py</code>	regenerates <code>MANIFEST.sha256</code> from the tracked file list	<code>MANIFEST.sha256</code>
<code>make_blinded.py</code>	strips identifiers and PDF metadata for the blinded copy	<code>manuscript_blinded.*</code>

Script	Purpose	Produces
<code>trace.py</code>	reviewer tool: any figure $\rightarrow$ its XBRL tags, frozen values and source URLs, with an optional live re-fetch	printed trail
<code>make_source_urls.py</code>	regenerates <code>data/SOURCE_URLS.txt</code>	<code>data/SOURCE_URLS.txt</code>
<code>reconcile.py</code>	(audit) matches constructed values to as-filed 10-K values for the five focal firms	reconciliation counts
<code>collect_*.py</code>	original data collection (Section 9; not run during reproduction)	<code>data/frozen/*</code>

6.2 Pipeline order

`verify.sh` runs the modules in dependency order; `claims.json` is timestamp-free, so re-runs are byte-identical:

```
make_claims→make_figure→build_gate→wordcount
```

7 Results (generated outputs)

Running the pipeline regenerates the following. In a clean checkout these already exist; the reproduction overwrites them and confirms they are unchanged.

Output	Contents
<code>claims.json</code>	every number the manuscript asserts. Keys used by the paper include <code>ladder</code> (Table 1), <code>placebo_ladder</code> (Table 2), <code>funnel</code> and <code>excluded</code> (the sample account), <code>composition</code> (the sectoral decomposition), <code>firm_level</code> and <code>leave_one_out_cum</code> (Table A3), <code>ai_filter</code> (Table A1 panel B), <code>alt_tag</code> , <code>annual</code> , <code>concentration</code> , <code>hashes</code>
<code>figures/Figure_1.pdf</code> , <code>.png</code>	(a) capital expenditure as a share of operating cash flow for the five largest investors; (b) Herfindahl index of capital expenditure within the 500, with and without those five; (c) coverage of the increment cumulated through each year
<code>manuscript.pdf</code>	the compiled submission (<code>reproduce.sh</code> only)

8 Reproduction and verification

`verify.sh` performs, and exits non-zero if any stage fails:

1. **Integrity.** Every archived file is checked against `MANIFEST.sha256`; a modified frozen input, script or manuscript fails here, before anything runs.
2. **Regeneration.** The analysis runs from `data/frozen/` only.
3. **Byte-identical re-run.** The regenerated `claims.json` and `figures/Figure_1.png` must be byte-for-byte identical to the committed ones; on a mismatch the script prints the exact claim path and both values.

4. **Manuscript audit.** `build_gate.py` reads every number out of `manuscript.tex` and requires each to be reproducible from `claims.json`. The abstract’s headline figures and every row of both tables are additionally anchored to their source, so a number swapped for a different valid one fails and is named. Exemptions — years, structural cuts, figures inside quoted 10-K text, and series identifiers — are listed explicitly in the source.
5. **Length.** The manuscript is checked against the journal’s word limit.

Determinism. The analysis has no stochastic component: no sampling, no bootstrap, no seed. `claims.json` is byte-identical across runs and machines. `Figure_1.png` is byte-identical in the archived reference environment (Python 3.11.5, `matplotlib==3.8.0`); raster output can depend on platform font and rendering details, so what `verify.sh` asserts is byte identity against the committed output in that environment. `Figure_1.pdf` embeds a build timestamp and is therefore excluded from the byte comparison; its pixel content is verified through the PNG.

Environment. Python 3.11.5 (standard library only for every reported number), `matplotlib==3.8.0` (figure only), and, for the PDF only, T_EX Live 2024 / pdfT_EX 1.40.26 with `elsarticle`, `siunitx`, `tabularx`, `booktabs`, `microtype`, `seqsplit`, `caption` and `lmodern`. Exact versions in `environment.txt`.

9 Data collection (not part of the reproduction)

`code/collect_*.py` document how the frozen inputs were originally collected: `collect_capex.py`, `collect_composition.py`, `collect_revenue.py` and `collect_alt_tags.py` pull the XBRL *frames* series; `collect_sic.py` pulls registrant classifications; `collect_ai_intensity.py` retrieves each firm’s most recent 10-K and scores it against a dictionary fixed before any financing result was computed. This phase requires internet access and is intentionally excluded from the reproducible core: the *frames* interface returns the most recently filed value for each fact, so a later collection would pick up subsequent filings or revised presentations. No API key is required at any point, and none is distributed. All reported results derive only from the frozen inputs.

10 Licensing and data availability

Code is released under the MIT Licence (see `LICENSE`); derived data, analysis outputs and text are released under CC-BY-4.0. The underlying raw data — SEC XBRL *frames*, SEC EDGAR filings and BLS series `WPUFD41312` — carry no additional restriction: SEC EDGAR public filing content is freely accessible and reusable, and BLS published material is in the U.S. public domain. The permanent citable archive is the Zenodo deposit at `10.5281/zenodo.22392416`, which reproduces every reported number, table and figure offline, deterministically, and at zero cost.